

Trust-weighted routing concentrates load, and cannot tell "untested" from "untrusted"

An empirical read of 3,742 live routing decisions on a Mininet testbed. The trust term that protects the fleet from a dishonest node also, by the same mechanism, permanently locks out a healthy one.

Run 498 s continuous Decisions 3,742 Outcomes 3,741 Fleet 4 edge nodes • 12 IoT clients

Controller os-ken / OpenFlow 1.3

Figures generated from the controller event bus (`data/events.jsonl`). Sections 1-4, 6 and the appendix describe one continuous 498 s run; Section 5 references an earlier 24-minute run and is labelled accordingly.

SUMMARY OF FINDINGS

F1 Half the fleet never received a single task

Two of four nodes served all 3,742 requests. The other two — one healthy, one hostile — were selected zero times. Capacity utilisation was 50% while every served task succeeded.

F2 The trust signal is self-reinforcing

Trust is earned only by completing tasks, and tasks are awarded by a score that weights trust most heavily. A node that starts fractionally behind can never catch up. `srv2` — healthy, fast, honest — sat frozen at its initial 0.500 for the entire run.

F3 Scoring and quarantine defend against opposite attacks

The score starves an attacker that looks bad; quarantine catches one that looks good. The sybil node was neutralised by the former and never triggered the latter — it was avoided, not detected.

F4 Three defects were measurement artefacts, not logic errors

Every false quarantine observed during development traced to comparing two quantities sampled on different time bases. The detector was sound; its inputs were not commensurable.

SECTION ONE

Method

The system under study is a trust-aware SDN load balancer. Twelve emulated IoT clients issue tasks to a virtual service address; an OpenFlow controller intercepts each new connection, selects an edge node, and

installs a rewrite rule pair steering that flow to the chosen server. Selection maximises a weighted score:

$\text{EdgeScore}(n) = w_1 \cdot \text{trust}(n) + w_2 \cdot (1 - \text{cpu}(n)) + w_3 \cdot (1 - \text{latency}(n))$, with weights tuned online by a UCB1 bandit over five fixed arms.

Trust is updated only on a completed task outcome. Nodes report CPU load over a health endpoint; the controller measures that endpoint's round-trip itself rather than trusting the node's self-reported figure.

One node, `srv3`, runs a **sybil** attack: it reports a fixed `cpu_load = 0.10` while burning four background CPU threads. One client, `iot12`, holds a deliberately wrong PRESENT-80 key. The remaining three nodes and eleven clients are honest.

All figures below are drawn from the controller's own event stream — routing decisions, per-second node snapshots, and switch flow counters — for one continuous 498-second run. Section 5 additionally references an earlier 24-minute run, and is labelled where it does.

FINDING ONE

Load concentrates on half the fleet

Across every one of 3,742 routing decisions, the controller chose `srv4` or `srv1`. It never once chose `srv2` or `srv3`.

3,742 DECISIONS	2 / 4 NODES EVER SELECTED	100% TASK SUCCESS RATE	0 DENIED ROUTINGS
--------------------	---------------------------------	---------------------------	----------------------

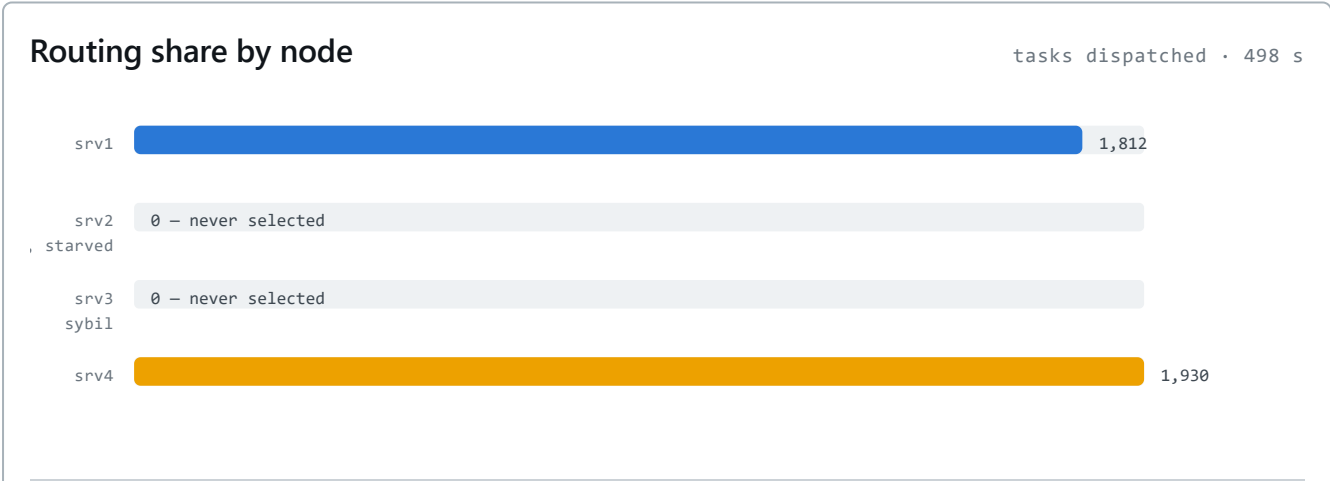


FIGURE 1

The split between `srv1` and `srv4` is near-even, and the remaining two nodes are not merely under-used but entirely unselected. `srv3` is the sybil; `srv2` is a fully healthy node.

Where each node placed in the ranking

share of 3,742 decisions by rank position

■ 1st (chosen) ■ 2nd ■ 3rd ■ 4th



FIGURE 2

srv2 reached second place in 585 decisions but first place in none — it was competitive throughout and still never won. srv3 placed last in 3,625 of 3,742. Rank is computed from the `ranked` array the controller records with every decision.

The distinction matters. A node that ranks last is being *correctly* avoided; a node that ranks second and never wins is being *starved*. Figure 2 separates the two cases, and the next section explains why the second one is stable rather than transient.

Trust compounds, so an early gap never closes

Trust changes only when a task completes. A node that is not selected therefore holds its score indefinitely — not decaying, not recovering, simply frozen at whatever it last was.

Trust trajectory

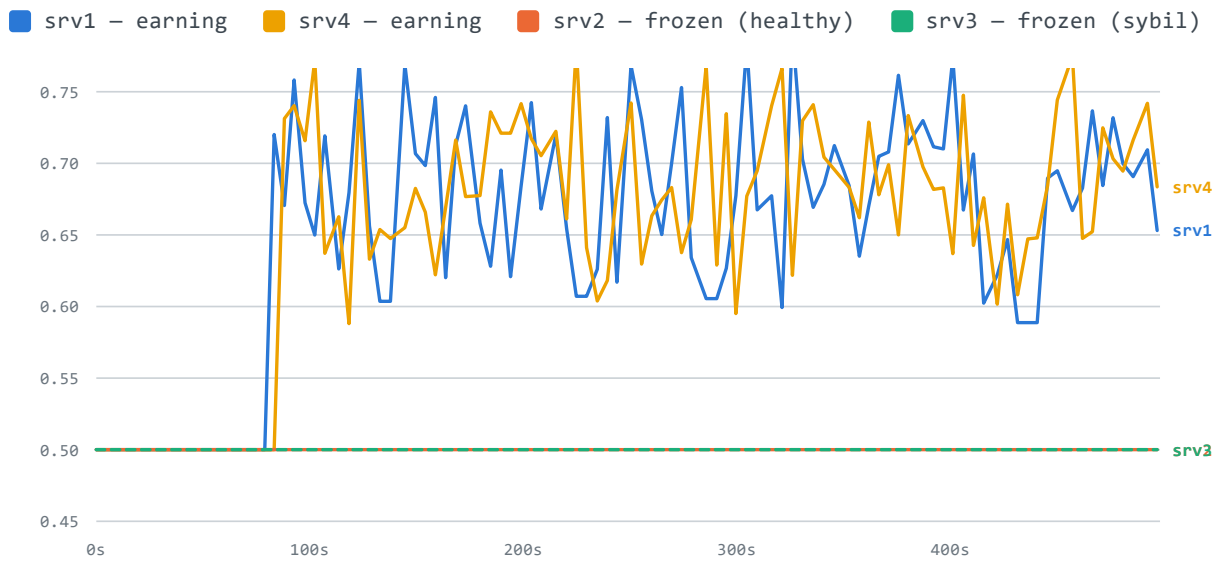
 $T(n) \cdot 0-1 \cdot \text{sampled per second}$


FIGURE 3

srv1 and srv4 climb from the 0.500 initial score to 0.65–0.68 as successful outcomes accumulate. srv2 and srv3 trace an identical flat line at exactly 0.500 for all 498 seconds — the two are superimposed. Neither ever completed a task, so neither ever had a trust update to apply.

EdgeScore separation over time

mean score per 100 s window

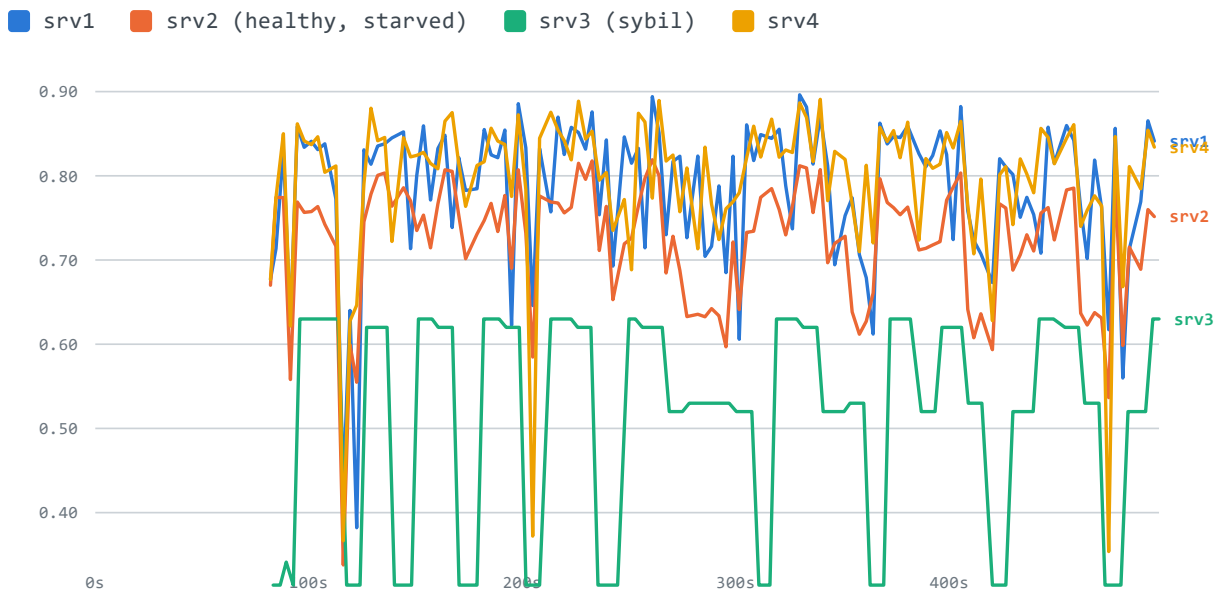


FIGURE 4

The mechanism in one picture. At the start of the run srv2 trails the leaders by roughly 0.04 — well within noise. As srv1 and srv4 convert task successes into trust, and trust is the heaviest-weighted term, the gap widens to 0.07–0.09 and stops being recoverable.

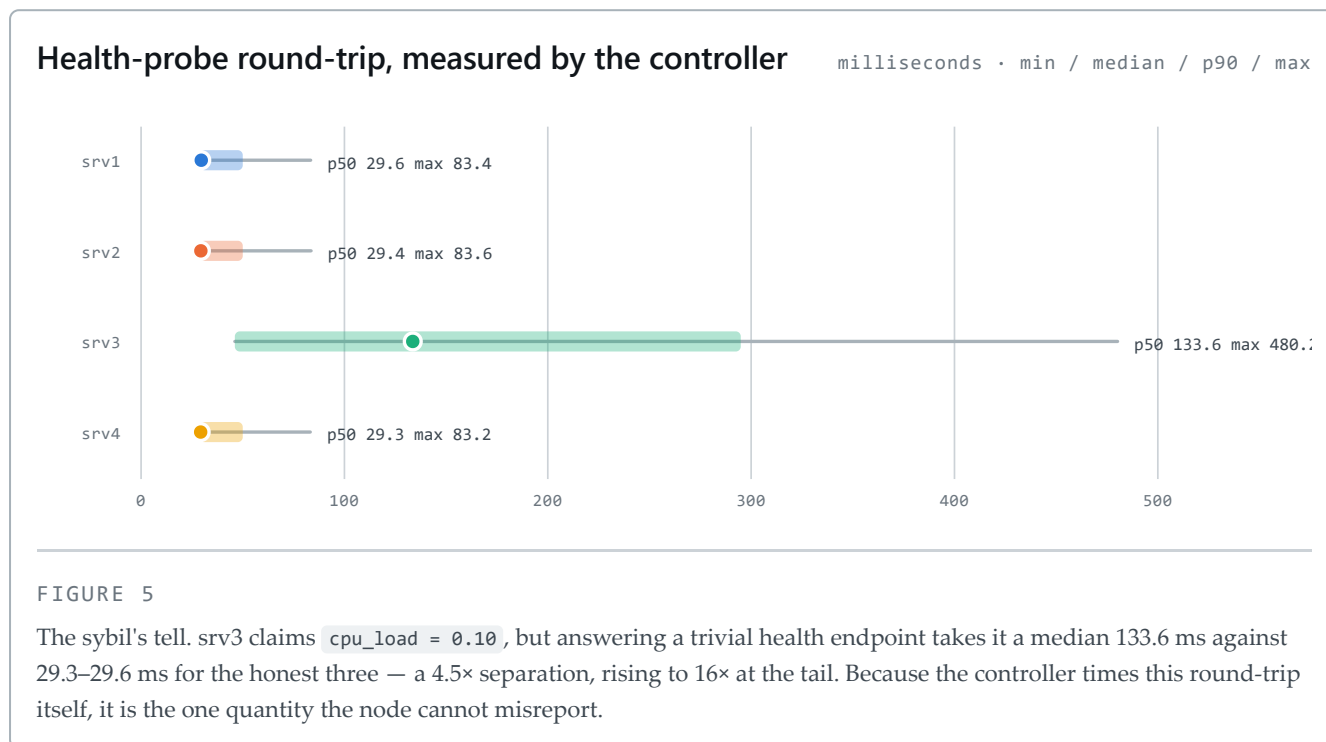
This is a preferential-attachment loop. Selection grants the only evidence that improves the score that governs selection, so a marginal early lead compounds into a permanent one. Three consequences follow.

- **Half the fleet's capacity is unavailable** while the system reports every node healthy and every task successful. Nothing in the telemetry flags a problem.
- **The score cannot represent uncertainty.** A node that has never been tried and a node that has been tried and found mediocre both sit at 0.500. The system has no way to express "unknown" as distinct from "average".
- **Starvation is indistinguishable from correct avoidance.** srv3 is avoided because it is genuinely slow; srv2 is avoided for no reason at all. Both appear in the telemetry as a node with 0.500 trust receiving no traffic.

The standard remedy is to make the selector occasionally sample nodes it does not favour, which is precisely the explore/exploit trade-off the project already solves with UCB1 — but applies to *weight tuning* rather than to *node selection*. The bandit optimises how to rank; it does not ensure the ranking is informed.

Two defences, aimed in opposite directions

The controller can exclude a node two ways: the score can rank it out of contention, or the anomaly gate can quarantine it outright. Which one engages depends on whether the attack makes the node look *attractive* or *unattractive*.



That measured latency feeds the score's third term, so the sybil sinks to last place and is starved of traffic. The attack fails — but it fails silently. Trust never collapses, because trust requires completed tasks, and it is given none. It ends the run at 0.500, indistinguishable from a node nobody happened to pick.

The reverse case is the packet-drop attack, which the same codebase supports. A dropping node accepts connections and never answers, so it reports genuinely idle CPU and answers its health probe fast — the most attractive profile on the board. It attracts maximum traffic, every task times out, and the timeout tell trips the anomaly gate within four outcomes. It cannot starve itself, because looking good is the whole attack.

The project's own threshold contract anticipates exactly this asymmetry: the anomaly gate exists because the trust formula "cannot isolate a node that lies about its load but still serves tasks well" — it floors around 0.44, never crossing the 0.3 isolation line. Scoring handles what it can see as bad; quarantine is the backstop for what looks good and behaves badly.

Quarantine and release events

498 s timeline · all four nodes

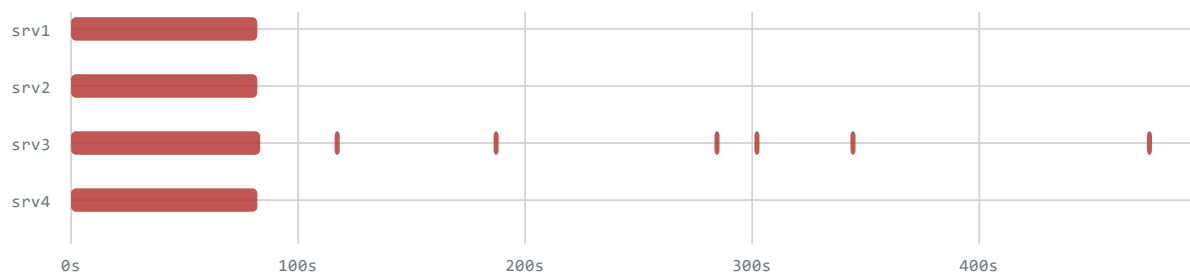


FIGURE 6

Every quarantine in this run was raised by an unreachable health probe, and every one was released automatically. The opening event at $t=0$ is all four nodes marked unreachable before their agents finish starting, cleared at $t=82$ s. The six later events are srv3 alone, each lasting about 1.1 s: under its own CPU load the sybil's health endpoint intermittently stalls past the probe timeout, then answers again.

The releases matter as much as the detections. Quarantine installs data-plane drop rules; if those rules also blocked the health probe, the signal that clears a node would be the signal quarantine suppresses, and every quarantine would be permanent. The 1.1-second round trips in Figure 6 are that recovery path working — nine raised, nine cleared, none stuck.

Sampling two quantities on different time bases

Reaching the run above required fixing three defects that presented as false quarantines. All three shared one root cause, and it is a measurement problem rather than a logic one. *This section draws on an earlier 24-minute run.*

The honesty check asks whether a node's claimed CPU matches the load the controller believes it has placed there. Both sides of that comparison were being sampled incorrectly.

The controller's side

Observed load was computed as `inflight / concurrency` read at the polling instant. With concurrency 4 that is a step function over {0, 0.25, 0.50, ...}. A node serving two concurrent requests at the sampled millisecond reads 0.50; against a truthful claim of 0.00 that is a 0.50 deviation, past the 0.40 threshold on its own. A healthy node was quarantined for it:

srv2 — quarantined on a single sample					earlier run
TIME	CLAIMED	OBSERVED	INFLIGHT	$\bar{A}$	STATE
18:02:40	0.00	0.00	0	0.00	ok
18:02:42	0.00	0.50	2	0.85	quarantined
18:02:43	0.00	0.00	0	0.98	quarantined

TABLE A

Two requests in flight for one instant. Little's Law describes a time-average, not an instantaneous queue depth; the fix integrates occupancy over a 3-second window.

The node's side

Averaging only the controller's half made it worse in the opposite direction. The agent samples `active / concurrency` the moment its handler runs, so its claim is a step function too. Once the controller's side was a 3-second integral and the node's side was still an instant, the check began firing the other way — a node caught mid-burst reporting 0.50 against a windowed 0.03.

Replaying all 1,320 recorded honesty trips from that run, averaging the node's claim over the same window avoids exactly three. Those three are the trips that started each cascade. The remaining 1,189 are downstream of the third defect.

The lock

The sweep that reclaims abandoned dispatches ran only when a new dispatch was registered. Once every node was quarantined, nothing registered — so the sweep stopped exactly when it was needed. Whatever was in flight at that moment stayed in flight: `inflight` froze at 3 for eight minutes, holding observed load at 0.75 against a truthful 0.00 and re-tripping the check every second. A transient false positive became terminal.

The general lesson is worth stating plainly, because it is not specific to this system: **a detector that compares two measurements is only as sound as their commensurability.** Each quantity here was individually correct. The defect was that one was an instantaneous sample and the other a time integral, and nothing in the type system or the tests said they could not be subtracted.

SECTION SIX

Discussion

Read together, the findings describe a system whose defensive properties are stronger than its distributive ones.

On defence the result is genuinely good. Across 3,742 decisions the dishonest node received no client traffic at all, and the unauthorised client was refused admission in 77 ms by the PRESENT-80 handshake, before any routing logic saw it. No task was ever served by a node that would have mishandled it. That the sybil was never formally quarantined is, arguably, the stronger claim: the defence held without needing to detect the attack, because the score is computed from a quantity the attacker does not control.

On distribution the same mechanism misfires. Trust-weighting assumes the score reflects a node's quality, but for an unsampled node it reflects only its history of not being sampled. The system conflates two very different statements — "this node performs at 0.5" and "this node has never been asked" — and the consequence is that half a healthy fleet sat idle for the entire run while the telemetry reported everything nominal.

Whether that trade is acceptable depends on the deployment. Where capacity is plentiful and hostility likely, starving anything suspicious is the right default. Where capacity is scarce, permanently idling 50% of it because of early noise is a serious cost — and one that would not appear in any dashboard the system currently produces.

SECTION SEVEN

Limitations

- **Single run, single topology.** Findings come from one 498-second run on one 4-node fleet. The starvation result in particular depends on initial conditions; a different early ordering may distribute differently, and repeated runs would be needed to establish how often `srv2`'s outcome occurs.
- **Offered load is low.** At roughly 7.5 tasks per second across four nodes, queue occupancy is near zero almost always. The CPU-honesty check depends on occupancy being informative, and at this load it is not — which is why the sybil's queue-based tell never fires and its latency tell does all the work.
- **One attack of each kind.** The drop-attack analysis in Section 4 is derived from the agent implementation and threshold contract, not observed in this run — no node was configured to drop during it.
- **Emulated timing.** Mininet link delays are synthetic and the controller shares a host with the data plane, so absolute latencies are not meaningful. Only the separation between nodes is, and that is what the argument rests on.

- **The observer is the system.** All measurements come from the controller's own event bus. Section 5 is a direct demonstration that this instrument can be wrong; the figures here postdate those fixes but are not independently corroborated.

APPENDIX

Data tables

Values underlying Figures 1–6, for readers who prefer the numbers to the marks.

PER-NODE SUMMARY · 498 S RUN

NODE	ROLE	TASKS	SUCCESS	TRUST (END)	RTT P50	RTT MAX
srv1	honest	1,812	1,812	0.6531	29.6 ms	83.4 ms
srv2	honest	0	—	0.5000	29.4 ms	83.6 ms
srv3	sybil	0	—	0.5000	133.6 ms	480.2 ms
srv4	honest	1,930	1,929	0.6835	29.3 ms	83.2 ms

RANK POSITION COUNTS

NODE	1ST (CHOSEN)	2ND	3RD	4TH
srv1	1,812	1,576	329	25
srv2	0	585	3,157	0
srv3	0	0	36	3,625
srv4	1,930	1,581	220	11

QUARANTINE / RELEASE EVENT LOG

T (S)	EVENT	NODE	TRUST	$\bar{A}$
0.0	quarantine	srv1	0.500	0.850
0.0	quarantine	srv2	0.500	0.850
0.0	quarantine	srv3	0.500	0.850
0.0	quarantine	srv4	0.500	0.850
82.0	released	srv1	0.500	0.150
82.0	released	srv2	0.500	0.150
82.0	released	srv4	0.500	0.150
83.3	released	srv3	0.500	0.150
116.1	quarantine	srv3	0.500	0.850
117.2	released	srv3	0.500	0.128
186.1	quarantine	srv3	0.500	0.850
187.3	released	srv3	0.500	0.128
283.4	quarantine	srv3	0.500	0.850
284.5	released	srv3	0.500	0.128
301.0	quarantine	srv3	0.500	0.850
302.2	released	srv3	0.500	0.128
343.3	quarantine	srv3	0.500	0.850
344.4	released	srv3	0.500	0.128
473.9	quarantine	srv3	0.500	0.850
475.0	released	srv3	0.500	0.128

Source: controller event bus, `data/events.jsonl`

Figures generated from 3,742 routing decisions, 3,741 task outcomes, 379 per-second node snapshots.